```c
#include <stdio.h>
#include <string.h>

struct item {
    float price;
    char name[100];
    int quantity;
};

struct item items_list[100];
int total_items = 0;

void display();

void accept() {
    printf("Enter the number of items: ");
    scanf("%d", &total_items);

    for (int i = 0; i < total_items; i++) {
        printf("\n Item %d \n", i + 1);
        printf("Name of item: ");
        scanf("%s", items_list[i].name);

        printf("Price of item: ");
        scanf("%f", &items_list[i].price);

        printf("Quantity of item: ");
        scanf("%d", &items_list[i].quantity);
    }
    display();
}

void display() {
    for (int i = 0; i < total_items; i++) {
        printf("Item %d:\n", i + 1);
        printf("Name: %s\n", items_list[i].name);
        printf("Price: Rs.%.2f\n", items_list[i].price);
        printf("Quantity: %d\n\n", items_list[i].quantity);}}

void bubbleSort() {
    struct item temp;
    for (int i = 0; i < total_items - 1; i++) {
        for (int j = 0; j < total_items - i - 1; j++) {
            if (items_list[j].price > items_list[j + 1].price) {
                temp = items_list[j];
                items_list[j] = items_list[j + 1];
                items_list[j + 1] = temp;}}}
```

```
    display();
}

void selectionSort() {
    struct item temp;
    int min_idx;
    for (int i = 0; i < total_items - 1; i++) {
        min_idx = i;
        for (int j = i + 1; j < total_items; j++) {
            if (items_list[j].price < items_list[min_idx].price) {
                min_idx = j;}}

        if (min_idx != i) {
            temp = items_list[i];
            items_list[i] = items_list[min_idx];
            items_list[min_idx] = temp;}}
    display();
}

void insertionSort() {
    struct item key;
    int j;
    for (int i = 1; i < total_items; i++) {
        key = items_list[i];
        j = i - 1;
        while (j >= 0 && items_list[j].price > key.price) {
            items_list[j + 1] = items_list[j];
            j = j - 1;}

        items_list[j + 1] = key;}
    display();
}

void shellSort() {
    struct item temp;
    for (int gap = total_items / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < total_items; i += 1) {
            temp = items_list[i];
            int j;
            for (j = i; j >= gap && items_list[j - gap].price >
temp.price; j -= gap) {
                items_list[j] = items_list[j - gap];
            }

            items_list[j] = temp;}}
    display();
}
```

```
void linearSearch(float target) {
    int found = 0;
    for (int i = 0; i < total_items; i++) {
        if (items_list[i].price == target) {
            printf("\n\nItem found at index %d:\n", i);
            printf("Name: %s\n", items_list[i].name);
            printf("Price: Rs.%.2f\n", items_list[i].price);
            printf("Quantity: %d\n\n", items_list[i].quantity);
            found = 1;}}

    if (!found) {
        printf("\n\nItem not found.\n");}}


int recBinarySearch(int low, int high, float target) {
    if (low > high) return -1;
    int mid = low + (high - low) / 2;
    if (items_list[mid].price == target) return mid;
    if (items_list[mid].price > target) return recBinarySearch(low, mid -
1, target);
    return recBinarySearch(mid + 1, high, target);
}

void nonrecBinarySearch(float target) {
    int low = 0, high = total_items - 1, found = 0;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (items_list[mid].price == target) {
            printf("\n\nItem found at index %d:\n", mid);
            printf("Name: %s\n", items_list[mid].name);
            printf("Price: Rs.%.2f\n", items_list[mid].price);
            printf("Quantity: %d\n\n", items_list[mid].quantity);
            found = 1;
            break;
        }

        if (items_list[mid].price < target) low = mid + 1;
        else high = mid - 1;
    }

    if (!found) {
        printf("\n\nItem not found.\n");}}

int main() {
    int option;
    float target;
```

```
    accept();

    do {
        printf("\nSelect an option from the following: ");
        printf("\n 1. Display.");
        printf("\n 2. Bubble Sort by Price.");
        printf("\n 3. Selection Sort by Price.");
        printf("\n 4. Insertion Sort by Price.");
        printf("\n 5. Shell Sort by Price.");
        printf("\n 6. Linear Search by Price.");
        printf("\n 7. Recursive Binary Search by Price (list must be
sorted).");
        printf("\n 8. Non-Recursive Binary Search by Price (list must be
sorted).");
        printf("\n 9. Exit.");
        printf("\nEnter your option: ");
        scanf("%d", &option);

        switch (option) {
            case 1:
                display();
                break;
            case 2:
                bubbleSort();
                break;
            case 3:
                selectionSort();
                break;
            case 4:
                insertionSort();
                break;
            case 5:
                shellSort();
                break;
            case 6:
                printf("Enter price to search: ");
                scanf("%f", &target);
                linearSearch(target);
                break;
            case 7:
                printf("Enter price to search: ");
                scanf("%f", &target);
            {
                int res = recBinarySearch(0, total_items - 1, target);
                if (res != -1) {
                    printf("\n\nItem found at index %d:\n", res);
```

```
                printf("Name: %s\n", items_list[res].name);
                printf("Price: Rs.%.2f\n", items_list[res].price);
                printf("Quantity: %d\n\n", items_list[res].quantity);
            } else {
                printf("\n\nItem not found.\n");}}
            break;
        case 8:
            printf("Enter price to search: ");
            scanf("%f", &target);
            nonrecBinarySearch(target);
            break;
        case 9:
            break;
        default:
            printf("Invalid.\n");
    }
} while (option != 9);

    return 0;
}
```

OUTPUT:

```
Name: soap
Price: Rs.20.00
Quantity: 5

Item 2:
Name: shampoo
Price: Rs.5.00
Quantity: 0


Select an option from the following:
 1. Display.
 2. Bubble Sort by Price.
 3. Selection Sort by Price.
 4. Insertion Sort by Price.
 5. Shell Sort by Price.
 6. Linear Search by Price.
 7. Recursive Binary Search by Price (list must be sorted).
 8. Non-Recursive Binary Search by Price (list must be sorted).
 9. Exit.
Enter your option: 5
Item 1:
Name: shampoo
```